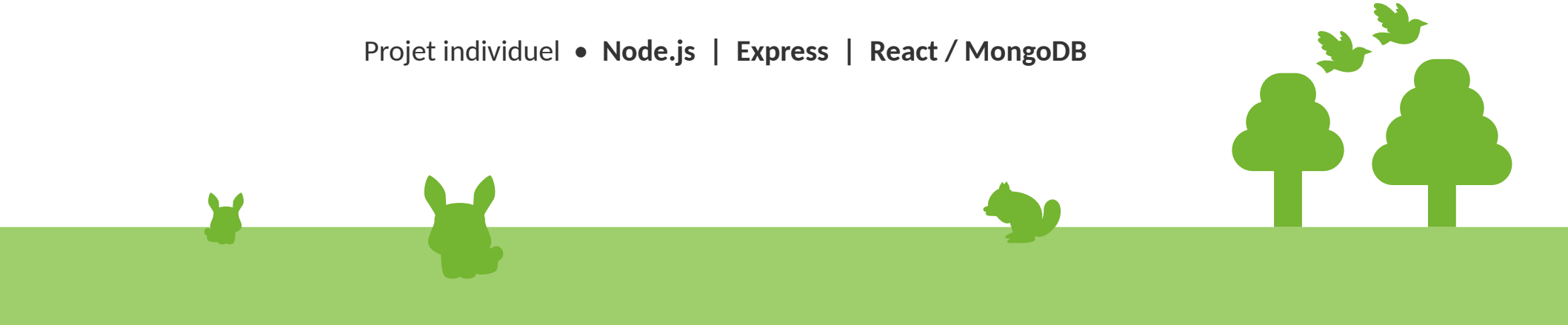
TaskFlow

Application Full Stack de gestion de tâches

Projet individuel • **Node.js | Express | React / MongoDB**



Sommaire

01 Contexte & Besoins

02 Choix techniques & Architecture

03 Conception & Modélisation

04 Développement Back-end

05 Sécurité & Tests

06 Interface Front-end

07 Difficultés & Solutions

08 Bilan & Compétences CDA

01

Contexte & Besoins

01 – Contexte & Besoin

Problème identifié

Absence d'outil centralisé pour suivre ses tâches quotidiennes avec catégories et indicateurs de progression.

Cible utilisateur

Tout utilisateur souhaitant organiser ses activités : étudiant, développeur, chef de projet.

User Stories principales

En tant qu'utilisateur, je veux m'inscrire et me connecter de façon sécurisée.

En tant qu'utilisateur, je veux créer, modifier et supprimer mes tâches.

En tant qu'utilisateur, je veux organiser mes tâches par catégorie et statut.

En tant qu'utilisateur, je veux consulter un tableau de bord avec mes statistiques.

02

Choix Techniques & Architecture

02 — Choix Techniques

Base de données

Mongoose/MongoDB

Scalabilité : D'abord indexation de la base via Mongoose si besoin de plus il faut un cluster MongoDB en Replica Set les bases secondaires pour la lecture et la base primaire pour l'écriture et la modification

Back-end

Node.js + Express

API REST légère, performante, JavaScript

Authentification

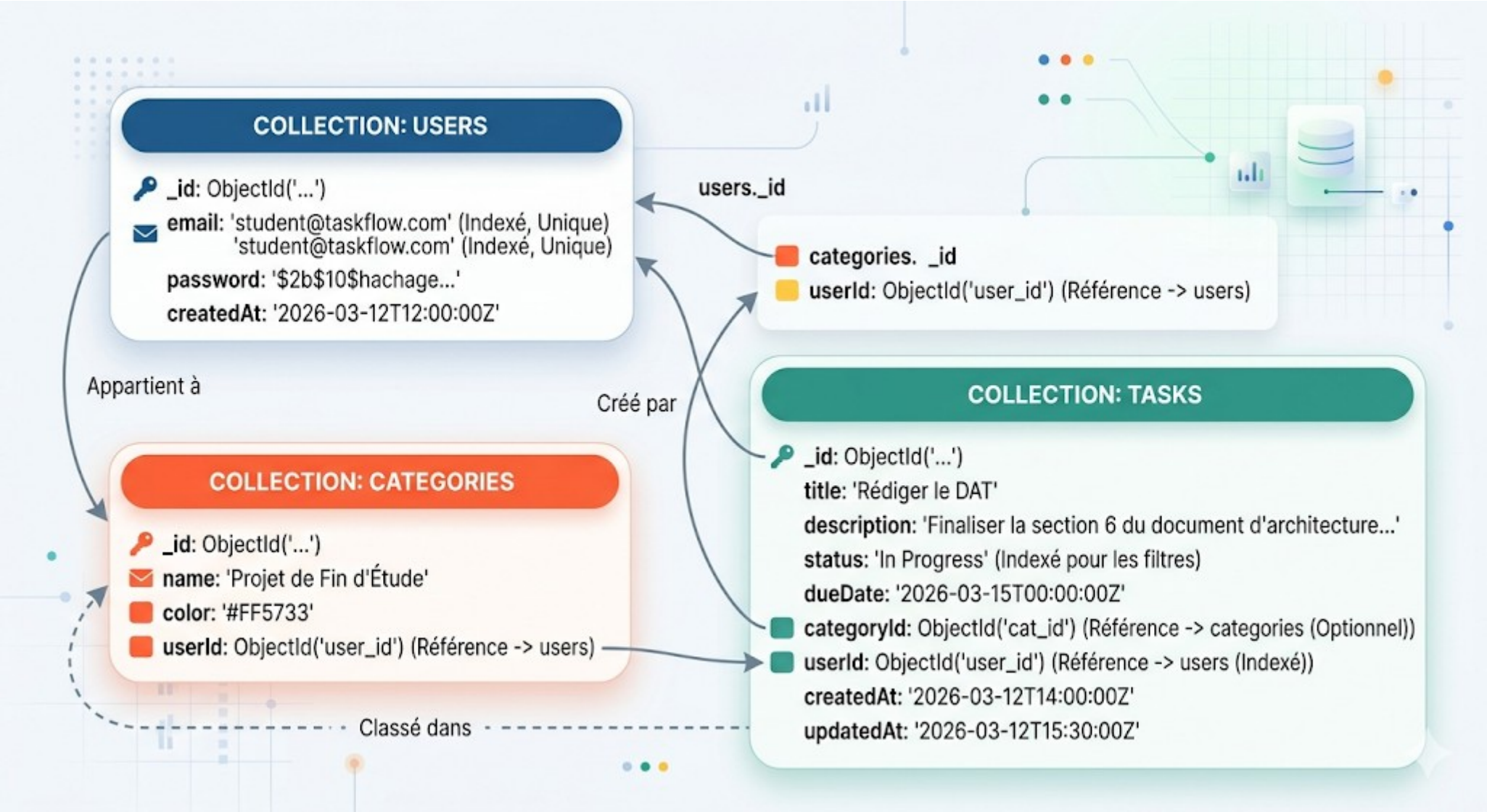
JWT + bcrypt

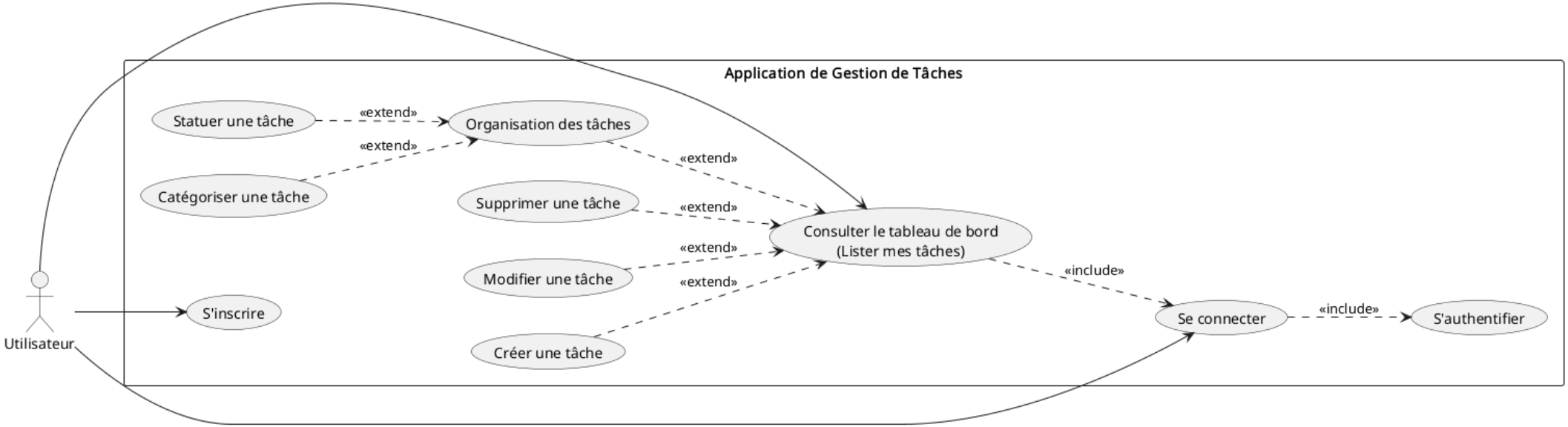
Permet une authentification stateless, idéale pour le scaling horizontal de l'application

03

Conception & Modélisation

Schéma MongoDB | Diagramme de cas d’utilisation


COLLECTION: USERS
_id: ObjectId('...')
email: 'student@taskflow.com' (Indexé, Unique)
'student@taskflow.com' (Indexé, Unique)
password: '$2b$10$hachage...'
createdAt: '2026-03-12T12:00:00Z'
users._id
categories. _id
userId: ObjectId('user_id') (Référence -> users)
Appartient à
Créé par
COLLECTION: TASKS
_id: ObjectId('...')
title: 'Rédiger le DAT'
description: 'Finaliser la section 6 du document d'architecture...'
status: 'In Progress' (Indexé pour les filtres)
dueDate: '2026-03-15T00:00:00Z'
categoryId: ObjectId('cat_id') (Référence -> categories (Optionnel))
userId: ObjectId('user_id') (Référence -> users (Indexé))
createdAt: '2026-03-12T14:00:00Z'
updatedAt: '2026-03-12T15:30:00Z'
COLLECTION: CATEGORIES
_id: ObjectId('...')
name: 'Projet de Fin d'Étude'
color: '#FF5733'
userId: ObjectId('user_id') (Référence -> users)
Classé dans


Application de Gestion de Tâches
Statuer une tâche
«extend»
Organisation des tâches
«extend»
Catégoriser une tâche
«extend»
Supprimer une tâche
«extend»
Consulter le tableau de bord
(Lister mes tâches)
«extend»
Modifier une tâche
«extend»
Créer une tâche
«include»
Se connecter
«include»
S'authentifier
S'inscrire
Utilisateur

04

Développement Back-end

API REST — Routes — Middlewares

04 — Architecture API REST

POST	/api/auth/register	Inscription utilisateur
POST	/api/auth/login	Connexion + retour JWT
GET	/api/tasks	Liste des tâches (auth requis)
POST	/api/tasks	Créer une tâche
PUT	/api/tasks/:id	Modifier une tâche
DELETE	/api/tasks/:id	Supprimer une tâche
GET	/api/categories	Liste des catégories

05

Sécurité & Qualité

JWT • Validation • Tests Jest

05 — Sécurité

Authentification JWT

- Token signé côté serveur
- Expiration configurable (1h)
- Middleware de vérification
- Refresh token (niveau 2)

Validation des données

- Joi / express-validator
- Contrôle de format email
- Longueur min. des mots de passe
- Sanitisation des entrées

Mots de passe

- bcrypt (salt rounds: 12)
- Jamais stocké en clair
- Comparaison constante time
- Hash vérifié à la connexion

Résultats Jest — Exemples de tests

```
Test Suites: 17 passed, 17 total
Tests:       88 passed, 88 total
Snapshots:   0 total
Time:        3.192 s
Ran all test suites.
```

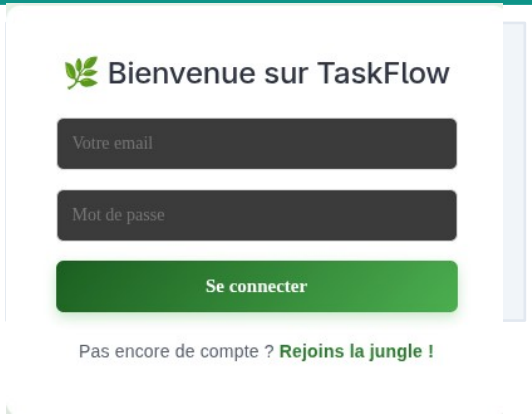
06

Interface Front-end

Pages — Intégration API — Dashboard

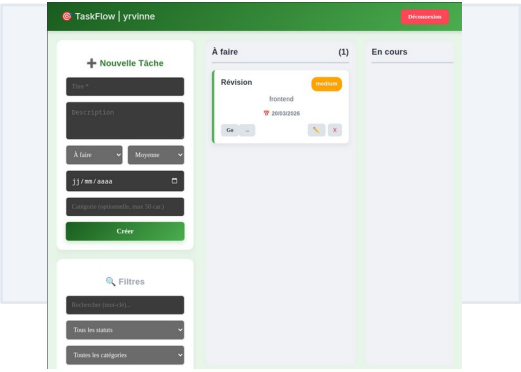
06 — Interface Utilisateur

Page Login



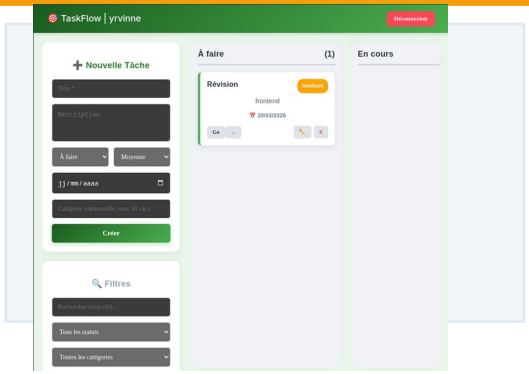
- Formulaire email + password
- Appel POST /api/auth/login
- Stockage du JWT (localStorage)
- Redirection vers dashboard

Liste des Tâches



- Fetch GET /api/tasks/user (avec token)
- Affichage dynamique en JS en SPA
- Filtres : statut / catégorie
- Boutons modifier / supprimer

Tableau de Bord



- Compteur tâches par statut
- Tâches en retard (due_date)

07

Difficultés & Solutions

Problèmes rencontrés — Approches retenues

07 — Difficultés rencontrées & Solutions apportées

01

Gestion des erreurs asynchrones

→ Mise en place d'un middleware centralisé avec try/catch et wrapAsync pour encapsuler les cas d'erreur de saisi avant le controller

02

L'expérience utilisateur face à l'expiration du Token JWT

→ Ajout d'une vérification dans chaque controller : la tâche demandée appartient-elle bien à l'utilisateur connecté (user_id / userId) ?

03

Beaucoup de couche de traitement

→ Développement TDD pour réduire les regression et le refactoring .

08

Bilan & Compétences CDA

Référentiel RNCP 37873 — Niveau 6

08 — Compétences CDA validées

Bloc 1 — Développer une application sécurisée

- Composants métier Node.js / Express
- Accès données Mongoose
- Auth JWT + bcrypt + validation

Bloc 2 — Concevoir et développer

- Architecture technique schéma
- Composants d'interface front-end
- Tests unitaires et d'intégration Jest

Bloc 3 — Préparer le déploiement

- Documentation README + Swagger
- Versionnement Git (commits réguliers)
- Déploiement

Axes d'amélioration : Déploiement Docker • Cluster database

Merci pour votre attention

Questions & Échanges

GitHub : github.com/YcafardyDiginamic/TaskFlow

Swagger : /api-docs

Node.js | Express | Mongoose / MongoDB | JWT

Jest | Git | Swagger | React